

Introduction to Programming: Python – Course 2

Mengzhuo Guo

Academic Year 2026-2027

Contents

1 Jupyter Notebook	2
2 Using Jupyter Notebook	2
2.1 Opening Jupyter Notebook	2
2.1.1 Method 1: Using Anaconda Navigator	2
2.1.2 Method 2: Using Anaconda Prompt	2
2.2 Creating a New Notebook	3
2.3 Renaming a Notebook	3
2.4 Understanding Notebook Cells	3
2.4.1 Code Cells	3
2.4.2 Markdown Cells	4
2.5 Running a Cell	4
2.6 Writing the First Python Programs	4
2.6.1 Printing Text	4
2.6.2 Printing Numbers	4
2.6.3 Basic Arithmetic	5
2.6.4 Using Variables	5
2.6.5 Printing Several Pieces of Information	5
2.6.6 Receiving User Input	6
2.7 Adding, Deleting, and Moving Cells	6
2.8 Saving a Notebook	6
2.9 Interrupting and Restarting the Kernel	7
2.10 The Importance of Execution Order	7
2.11 Reading Error Messages	7
2.12 Closing Jupyter Notebook Correctly	8
2.13 In-Class Practice	8
2.14 Summary	9
3 Python Basic Syntax, Variables, and Data Types	10
3.1 Basic Python Syntax	10
3.1.1 Indentation	10
3.2 Variables	10
3.3 Constants	11
3.4 Identifiers and Naming Rules	11
3.4.1 Recommended Naming Style	12
3.5 Basic Data Types	12
3.5.1 Integer	12
3.5.2 Floating-Point Number	12
3.5.3 String	12
3.5.4 Boolean	13
3.5.5 None	13
3.6 Checking Data Types	13
3.7 Comments	13
3.8 Output with <code>print()</code>	14

3.8.1	Formatted Strings	14
3.8.2	Escape Characters	14
3.9	Controlling the Output of <code>print()</code>	15
3.10	Input with <code>input()</code>	16
3.11	Basic Type Conversion	16
3.12	Basic Operators	16
3.12.1	Arithmetic Operators	16
3.12.2	Comparison Operators	17
3.12.3	Assignment Operators	17
3.12.4	Logical Operators	18
3.13	Expressions	18
3.13.1	Operator Precedence	18
3.14	A Complete Simple Program	19
3.15	Common Beginner Mistakes	19
3.15.1	Forgetting Quotation Marks	19
3.15.2	Using <code>=</code> Instead of <code>==</code>	19
3.15.3	Using Numerical Input Without Conversion	19
3.15.4	Incorrect Capitalization	20
3.16	Quick Practice	20
3.17	Summary	20

1 Jupyter Notebook

Anaconda provides both the Python programming language and several useful development tools. In this course, we mainly use **Jupyter Notebook** to write and run Python programs.

A Jupyter Notebook allows us to divide a program into small blocks called **cells**. We can run one cell at a time and immediately see the result. Therefore, Jupyter Notebook is especially suitable for beginners, classroom demonstrations, data analysis, and experimentation.

2 Using Jupyter Notebook

2.1 Opening Jupyter Notebook

There are two common ways to open Jupyter Notebook.

2.1.1 Method 1: Using Anaconda Navigator

1. Open **Anaconda Navigator** from the Windows Start menu.
2. Find the **Jupyter Notebook** application.
3. Click the **Launch** button.
4. A browser window will open automatically.

The browser page displays the files and folders on the computer. This page is called the Jupyter Notebook dashboard.

2.1.2 Method 2: Using Anaconda Prompt

Open **Anaconda Prompt** and enter the following command:

```
jupyter notebook
```

Press the **Enter** key. Jupyter Notebook will start, and a browser window will normally open automatically.

If you have created a separate Conda environment, activate the environment before starting Jupyter Notebook:

```
conda activate environment_name
jupyter notebook
```

For example, if the environment is called `python_course`, use:

```
conda activate python_course
jupyter notebook
```

This helps ensure that the Notebook uses the Python packages installed in the correct environment.

2.2 Creating a New Notebook

After opening the Jupyter Notebook dashboard, follow these steps:

1. Open the folder in which you want to save your work.
2. Click the **New** button in the upper-right corner.
3. Select a Python kernel, such as **Python 3**.
4. A new Notebook will open in another browser tab.

A newly created Notebook usually has a name such as:

Untitled.ipynb

The file extension `.ipynb` means that the file is a Jupyter Notebook file.

2.3 Renaming a Notebook

It is important to give every Notebook a meaningful name.

To rename a Notebook:

1. Click the current file name at the top of the page.
2. Enter a new file name.
3. Click **Rename**.

For example, the first Notebook in this course can be named:

course1_basic_python.ipynb

Do not use special symbols in the file name. It is recommended to use English letters, numbers, hyphens, or underscores.

2.4 Understanding Notebook Cells

A Notebook consists of multiple cells. The two most commonly used cell types are:

- **Code cell**: used to write and run Python code.
- **Markdown cell**: used to write titles, explanations, notes, formulas, and documentation.

The current cell type can be selected from the toolbar.

2.4.1 Code Cells

A code cell accepts Python commands. For example, enter the following code into a cell:

```
print("Hello, Python!")
```

Run the cell to see the output:

Hello, Python!

2.4.2 Markdown Cells

A Markdown cell is used to write formatted text rather than Python code.

For example:

```
# My First Python Notebook
```

This Notebook records my first Python programs.

After running the Markdown cell, the text will be displayed as a formatted title and paragraph.

Common Markdown syntax includes:

Markdown syntax	Meaning
# Title	Main title
## Subtitle	Second-level title
text	Bold text
text	Italic text
- item	Bullet point

2.5 Running a Cell

There are several ways to run a cell.

- Click the **Run** button on the toolbar.
- Press **Shift + Enter** to run the current cell and move to the next cell.
- Press **Ctrl + Enter** to run the current cell and remain in the same cell.
- Press **Alt + Enter** to run the current cell and create a new cell below it.

For beginners, **Shift + Enter** is the most frequently used shortcut.

When a code cell is running, the symbol on the left side may appear as:

In [*]:

After the cell finishes running, the asterisk will normally be replaced by a number, such as:

In [1]:

The number indicates the order in which the cells were executed.

2.6 Writing the First Python Programs

2.6.1 Printing Text

The `print()` function displays information on the screen.

```
print("Welcome to the Python course.")  
print("My name is Mengzhuo Guo.")
```

Students can replace the second sentence with their own names.

2.6.2 Printing Numbers

Python can also print numbers:

```
print(2026)  
print(3.14)
```

Text must normally be placed inside quotation marks, but numbers do not need quotation marks.

Compare the following two commands:

```
print("2026")  
print(2026)
```

Although the displayed results look similar, the first value is text and the second value is a number.

2.6.3 Basic Arithmetic

Python can be used as a calculator.

```
print(10 + 5)
print(10 - 5)
print(10 * 5)
print(10 / 5)
```

The main arithmetic operators are shown below.

Operator	Meaning	Example
+	Addition	10 + 5
-	Subtraction	10 - 5
*	Multiplication	10 * 5
/	Division	10 / 5
//	Integer division	10 // 3
%	Remainder	10 % 3
**	Exponentiation	2 ** 3

Try the following code:

```
print(10 // 3)
print(10 % 3)
print(2 ** 3)
```

2.6.4 Using Variables

A variable is a name used to store a value.

```
student_name = "Alice"
student_id = "20260001"
age = 18
```

```
print(student_name)
print(student_id)
print(age)
```

The equal sign = assigns the value on the right-hand side to the variable on the left-hand side.

A variable can be used in a calculation:

```
price = 20
quantity = 3
total_cost = price * quantity

print(total_cost)
```

The output is:

60

2.6.5 Printing Several Pieces of Information

The print() function can display several values together.

```
name = "Alice"
student_id = "20260001"

print("Name:", name)
print("Student ID:", student_id)
```

Students can also write:

```
name = "Alice"
student_id = "20260001"
course = "Python Language and Applications"

print("My name is", name)
print("My student ID is", student_id)
print("My course is", course)
```

2.6.6 Receiving User Input

The `input()` function allows a program to receive information from the user.

```
name = input("Please enter your name: ")
print("Hello,", name)
```

When the cell is executed, Python waits for the user to enter a value.

A value returned by `input()` is treated as text. To use an input value in a numerical calculation, it should be converted into a number.

```
age = int(input("Please enter your age: "))
print("Next year, you will be", age + 1)
```

Here, `int()` converts the input into an integer.

2.7 Adding, Deleting, and Moving Cells

Common cell operations include:

- Click the **+** button to add a new cell.
- Use the up and down arrow buttons to move a cell.
- Select a cell and use **Edit** → **Delete Cells** to delete it.
- Use **Edit** → **Copy Cells** and **Edit** → **Paste Cells** to copy and paste cells.

In command mode, some common keyboard shortcuts are:

Shortcut	Function
A	Insert a cell above
B	Insert a cell below
D, D	Delete the selected cell
M	Change the cell to Markdown
Y	Change the cell to Code
Z	Undo cell deletion

The shortcuts above are available when the Notebook is in **command mode**. Press **Esc** to enter command mode.

Press **Enter** to return to **edit mode**, in which the content of a cell can be edited.

2.8 Saving a Notebook

Jupyter Notebook usually saves files automatically. However, students should also save their work manually.

To save a Notebook:

- Click the save icon on the toolbar.
- Select **File** → **Save and Checkpoint**.
- Press **Ctrl + S**.

Before closing the browser, check that the Notebook has been saved.

2.9 Interrupting and Restarting the Kernel

The **kernel** is the program that executes Python code.

Sometimes a program may run for a long time or fail to stop. For example:

```
while True:
    print("Running")
```

This is an infinite loop. To stop it, select:

Kernel -> Interrupt

If the Notebook behaves abnormally, select:

Kernel -> Restart

Restarting the kernel clears all variables stored in memory. Therefore, the cells may need to be executed again from the beginning.

A useful option is:

Kernel -> Restart and Run All

This restarts the kernel and executes all cells in order.

2.10 The Importance of Execution Order

The output of a Notebook depends on the order in which the cells are executed.

Consider the following two cells.

Cell 1:

```
number = 10
```

Cell 2:

```
print(number)
```

Cell 1 must be executed before Cell 2. Otherwise, Python may report the following error:

```
NameError: name 'number' is not defined
```

Therefore, students should organize their code from top to bottom and execute the cells in a logical order.

2.11 Reading Error Messages

Errors are a normal part of programming. An error message usually provides useful information about the problem.

For example:

```
print("Hello)
```

Python reports a syntax error because the quotation mark is not closed.

Another example is:

```
print(student_name)
```

If `student_name` has not been defined, Python reports a `NameError`.

When an error occurs:

1. Read the last line of the error message.
2. Identify the type of error.
3. Check the indicated line of code.
4. Correct the code and run the cell again.

Do not be afraid of error messages. Learning to understand and correct errors is an important programming skill.

2.12 Closing Jupyter Notebook Correctly

Closing the browser tab does not always stop the Jupyter Notebook server.

To close Jupyter Notebook correctly:

1. Save the Notebook.
2. Return to the Jupyter dashboard.
3. Select the running Notebook.
4. Click **Shutdown**.
5. Return to Anaconda Prompt and press **Ctrl + C**.
6. Confirm that the Jupyter server should be stopped.

If Jupyter Notebook was opened through Anaconda Navigator, it can also be stopped from Anaconda Navigator.

2.13 In-Class Practice

Complete the following tasks in a new Jupyter Notebook.

Task 1: Personal Information

Create three variables to store your name, student ID, and major. Print them in the following format:

```
Name: Alice
Student ID: 20260001
Major: Business Administration
```

Task 2: Basic Calculation

Suppose a student buys three books. Each book costs 25 yuan. Calculate and print the total cost.

```
book_price = 25
number_of_books = 3
```

The expected output is:

```
Total cost: 75
```

Task 3: Rectangle Area

Create two variables, `length` and `width`, and calculate the area of a rectangle.

For example:

```
length = 10
width = 6
```

The expected output is:

```
The area is 60
```

Task 4: User Input

Write a program that asks the user to enter a name and then prints a welcome message.

An example interaction is:

```
Please enter your name: Alice
Welcome to Python, Alice!
```

Task 5: Python and Environment Information

Run the following program to display the Python version:

```
import sys

print("Python version:")
print(sys.version)
```

Run the following program to display the current Conda environment:

```
import os

environment_name = os.environ.get("CONDA_DEFAULT_ENV")
print("Conda environment:", environment_name)
```

Finally, combine the information into one program:

```
import sys
import os

name = "Your Name"
student_id = "Your Student ID"
environment_name = os.environ.get("CONDA_DEFAULT_ENV")

print("Name:", name)
print("Student ID:", student_id)
print("Python version:", sys.version)
print("Conda environment:", environment_name)
```

Replace `Your Name` and `Your Student ID` with your own information.

2.14 Summary

After completing this section, students should be able to:

- open Jupyter Notebook using Anaconda Navigator or Anaconda Prompt;
- create, rename, save, and close a Notebook;
- distinguish between code cells and Markdown cells;
- run Python code cell by cell;
- use `print()`, variables, arithmetic operations, and `input()`;
- understand the basic meaning of the kernel and execution order;
- read and correct simple Python error messages.

3 Python Basic Syntax, Variables, and Data Types

This section provides a brief introduction to the basic syntax of Python. The purpose is not to fully understand or memorize every rule. Instead, students should become familiar with how a simple Python program looks and how basic statements are written.

The topics include:

- Basic Python syntax
- Variables and constants
- Identifiers and naming rules
- Basic data types
- Comments
- Input and output
- Basic operators and expressions

3.1 Basic Python Syntax

Python programs are usually written one statement per line.

For example:

```
print("Hello, Python!")
print("Welcome to the Python course.")
```

The `print()` function displays information on the screen.

Unlike some programming languages, Python does not normally require a semicolon at the end of a statement.

```
print("Hello")           # Recommended
print("Hello");          # Possible, but unnecessary
```

3.1.1 Indentation

Python uses indentation to organize code blocks. Indentation means adding spaces at the beginning of a line.

For example:

```
age = 18

if age >= 18:
    print("You are an adult.")
```

The indented statement belongs to the `if` block.

A common convention is to use four spaces for one level of indentation.

Important: Incorrect indentation may cause an error or change the meaning of the program.

3.2 Variables

A variable is a name used to store a value.

The basic form of variable assignment is:

`variable_name = value`

For example:

```
name = "Alice"
age = 18
height = 1.68
```

Here:

- `name` stores the text "Alice".
- `age` stores the integer 18.
- `height` stores the decimal number 1.68.

The symbol `=` means assignment in Python. It assigns the value on the right to the variable on the left.

```
score = 80
score = 90

print(score)
```

The output is:

```
90
```

The second assignment replaces the previous value.

3.3 Constants

A constant is a value that is not expected to change during a program.

Python does not strictly prevent a constant from being changed. Programmers usually write constant names using uppercase letters.

```
PI = 3.14159
MAX_SCORE = 100
COURSE_NAME = "Python Language and Applications"
```

Uppercase names tell other programmers that these values should normally not be modified.

Note: Uppercase naming is a programming convention, not a rule enforced by Python.

3.4 Identifiers and Naming Rules

An identifier is a name used for a variable, function, class, or other object in a program.

Examples of identifiers include:

```
student_name = "Alice"
student_age = 18
final_score = 95
```

Python identifiers follow several basic rules:

1. An identifier may contain letters, numbers, and underscores.
2. An identifier cannot begin with a number.
3. Spaces are not allowed inside an identifier.
4. Python identifiers are case-sensitive.
5. Python keywords cannot be used as identifiers.

Examples are shown below.

Identifier	Valid?	Explanation
<code>student_name</code>	Yes	Uses letters and an underscore
<code>score2</code>	Yes	A number may appear after a letter
<code>2score</code>	No	Cannot begin with a number
<code>student name</code>	No	Spaces are not allowed
<code>class</code>	No	<code>class</code> is a Python keyword

Python is case-sensitive. Therefore, the following are three different variables:

```
score = 90
Score = 80
SCORE = 70
```

3.4.1 Recommended Naming Style

Variable names should be clear and meaningful.

Python programmers commonly use lowercase words separated by underscores. This style is called *snake case*.

```
student_name = "Alice"
exam_score = 92
course_average = 85.5
```

Avoid unclear names when possible.

```
# Less clear
a = 18
x = 92

# More clear
student_age = 18
exam_score = 92
```

3.5 Basic Data Types

A data type describes the kind of value stored in a variable.

Some common Python data types are shown below.

Data Type	Python Name	Example	Meaning
Integer	<code>int</code>	18	Whole number
Floating-point number	<code>float</code>	3.14	Decimal number
String	<code>str</code>	"Python"	Text
Boolean	<code>bool</code>	True	True or false value
None value	<code>NoneType</code>	None	No value

3.5.1 Integer

An integer is a whole number without a decimal point.

```
age = 18
number_of_students = 40
temperature = -5
```

3.5.2 Floating-Point Number

A floating-point number contains a decimal point.

```
height = 1.68
price = 19.99
average_score = 86.5
```

3.5.3 String

A string is a sequence of characters used to represent text.

Strings can be written using single or double quotation marks.

```
name = "Alice"
course = 'Python'
message = "Welcome to the course!"
```

Quotation marks are part of the Python syntax, but they are not normally displayed when the string is printed.

```
message = "Hello"
print(message)
```

The output is:

```
Hello
```

3.5.4 Boolean

A Boolean value represents one of two logical states:

- True
- False

For example:

```
is_student = True
is_finished = False
```

The first letters of **True** and **False** must be uppercase.

3.5.5 None

The value **None** means that no value is currently available.

```
result = None
print(result)
```

It is often used as a temporary or empty value.

3.6 Checking Data Types

The `type()` function can be used to check the data type of a value.

```
age = 18
height = 1.68
name = "Alice"
is_student = True

print(type(age))
print(type(height))
print(type(name))
print(type(is_student))
```

The output is similar to:

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

3.7 Comments

Comments are notes written inside a program. Python ignores comments when it runs the program.

A single-line comment begins with the symbol `#`.

```
# This is a comment

name = "Alice" # Store the student's name

print(name)
```

Comments can help explain:

- What the program does
- Why a statement is needed
- What a variable represents

Comments should be useful and concise.

```
# Calculate the total price
total_price = price * quantity
```

3.8 Output with print()

The `print()` function displays values on the screen.

```
print("Hello, Python!")
print(100)
print(3.14)
```

Several values can be printed together by separating them with commas.

```
name = "Alice"
age = 18

print("Name:", name)
print("Age:", age)
```

The output is:

```
Name: Alice
Age: 18
```

3.8.1 Formatted Strings

An f-string provides a convenient way to include variables inside a string.

```
name = "Alice"
score = 95

print(f"{name}'s score is {score}.")
```

The output is:

```
Alice's score is 95.
```

3.8.2 Escape Characters

Some special characters cannot be typed directly in an ordinary string. Python uses a backslash `\` followed by another character to represent them. These combinations are called *escape characters*.

Two commonly used escape characters are:

Escape Character	Meaning	Description
<code>\n</code>	Newline	Move the following text to a new line
<code>\t</code>	Tab	Insert horizontal spacing between values

For example:

```
print("Name: Alice\nAge: 18\nScore: 95")
```

The output is:

```
Name: Alice
Age: 18
Score: 95
```

3.9 Controlling the Output of `print()`

By default, the `print()` function moves to a new line after displaying its content.

```
print("Hello") print("Python")
```

The output is:

```
Hello Python
```

The `end` parameter controls what is printed at the end of a `print()` statement.

For example, the following code prevents Python from moving to a new line:

```
print("Hello", end=" ")
print("Python")
```

The output is:

```
Hello Python
```

The `end` parameter can also use other characters.

```
print("Python", end="---")
print("Course")
```

The output is:

```
Python---Course
```

When several values are printed together, Python separates them with spaces by default.

```
print("Alice", 18, 95)
```

The output is:

```
Alice 18 95
```

The `sep` parameter changes the separator between values.

```
print("Alice", 18, 95, sep=", ")
```

The output is:

```
Alice, 18, 95
```

Another example is:

```
year = 2026
month = 9
day = 1
print(year, month, day, sep="-")
```

The output is:

```
2026-9-1
```

The `sep` and `end` parameters can be used together.

```
print("Name", "Age", "Score", sep="\t", end="\n")
print("Alice", 18, 95, sep="\t")
```

The output is similar to:

```
Name Age Score
Alice 18 95
```

3.10 Input with input()

The `input()` function allows a user to enter information.

```
name = input("Please enter your name: ")
print("Hello,", name)
```

The message inside `input()` is called a prompt.

The value returned by `input()` is always a string.

For example:

```
age = input("Please enter your age: ")

print(age)
print(type(age))
```

Even when the user enters 18, Python treats it as the string "18".

3.11 Basic Type Conversion

Type conversion changes a value from one data type to another.

Common conversion functions include:

Function	Purpose
<code>int()</code>	Convert a value to an integer
<code>float()</code>	Convert a value to a floating-point number
<code>str()</code>	Convert a value to a string
<code>bool()</code>	Convert a value to a Boolean value

For example:

```
age_text = "18"
age_number = int(age_text)

print(age_number)
print(type(age_number))
```

When numerical input is required, type conversion is usually necessary.

```
age = int(input("Please enter your age: "))
print("Next year, you will be", age + 1)
```

3.12 Basic Operators

Operators are symbols or words used to perform operations on values.

3.12.1 Arithmetic Operators

Operator	Meaning	Example	Result
<code>+</code>	Addition	<code>5 + 2</code>	7
<code>-</code>	Subtraction	<code>5 - 2</code>	3
<code>*</code>	Multiplication	<code>5 * 2</code>	10
<code>/</code>	Division	<code>5 / 2</code>	2.5
<code>//</code>	Floor division	<code>5 // 2</code>	2
<code>%</code>	Remainder	<code>5 % 2</code>	1
<code>**</code>	Exponentiation	<code>5 ** 2</code>	25

Example:

```

a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)

```

3.12.2 Comparison Operators

Comparison operators compare two values. The result is either **True** or **False**.

Operator	Meaning	Example
==	Equal to	x == 10
!=	Not equal to	x != 10
>	Greater than	x > 10
<	Less than	x < 10
>=	Greater than or equal to	x >= 10
<=	Less than or equal to	x <= 10

Example:

```

score = 85

print(score == 85)
print(score > 60)
print(score < 60)

```

3.12.3 Assignment Operators

The basic assignment operator is =.

```
score = 80
```

Python also provides shorter assignment forms.

Operator	Example	Equivalent Statement
+=	x += 2	x = x + 2
-=	x -= 2	x = x - 2
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2

Example:

```

score = 80
score += 5

print(score)

```

The output is:

```
85
```

3.12.4 Logical Operators

Logical operators combine or modify Boolean expressions.

Operator	Meaning	Example
and	Both conditions are true	age >= 18 and age <= 60
or	At least one condition is true	score < 60 or absent
not	Reverse a Boolean value	not is_finished

Example:

```
age = 20
has_ticket = True

print(age >= 18 and has_ticket)
```

3.13 Expressions

An expression is a combination of values, variables, and operators that produces a result.

Examples include:

```
5 + 3
price * quantity
score >= 60
first_name + last_name
```

An expression can be assigned to a variable.

```
price = 20
quantity = 3

total_price = price * quantity

print(total_price)
```

3.13.1 Operator Precedence

Python generally follows the usual mathematical order of operations:

1. Parentheses
2. Exponentiation
3. Multiplication, division, floor division, and remainder
4. Addition and subtraction
5. Comparison operations
6. Logical operations

For example:

```
result1 = 2 + 3 * 4
result2 = (2 + 3) * 4

print(result1)
print(result2)
```

The output is:

```
14
20
```

Using parentheses can make an expression easier to understand.

3.14 A Complete Simple Program

The following program combines variables, input, output, type conversion, operators, and expressions.

```
# Ask the user for basic information
name = input("Please enter your name: ")
student_id = input("Please enter your student ID: ")
score1 = float(input("Please enter your first score: "))
score2 = float(input("Please enter your second score: "))

# Calculate the average score
average_score = (score1 + score2) / 2

# Display the result
print("Name:", name)
print("Student ID:", student_id)
print("Average score:", average_score)
print("Passed:", average_score >= 60)
```

This program demonstrates the following basic ideas:

- Variables store information.
- `input()` receives information from the user.
- `float()` converts text into a number.
- Operators perform calculations and comparisons.
- `print()` displays the results.
- Comments explain the purpose of the code.

3.15 Common Beginner Mistakes

3.15.1 Forgetting Quotation Marks

Incorrect:

```
name = Alice
```

Correct:

```
name = "Alice"
```

3.15.2 Using = Instead of ==

Assignment:

```
score = 80
```

Comparison:

```
score == 80
```

3.15.3 Using Numerical Input Without Conversion

Incorrect:

```
age = input("Enter your age: ")
next_age = age + 1
```

Correct:

```
age = int(input("Enter your age: "))
next_age = age + 1
```

3.15.4 Incorrect Capitalization

Incorrect:

```
is_student = true
```

Correct:

```
is_student = True
```

3.16 Quick Practice

Try to write a short Python program that:

1. Asks the user to enter a name.
2. Asks the user to enter two numbers.
3. Calculates the sum and average of the two numbers.
4. Prints the name, sum, and average.
5. Checks whether the average is greater than or equal to 60.

A possible solution is:

```
name = input("Enter your name: ")
number1 = float(input("Enter the first number: "))
number2 = float(input("Enter the second number: "))

total = number1 + number2
average = total / 2

print("Name:", name)
print("Sum:", total)
print("Average:", average)
print("Average is at least 60:", average >= 60)
```

3.17 Summary

In this section, we briefly introduced the basic appearance and syntax of Python programs.

Students should now be familiar with the following ideas:

- Python uses indentation to organize code.
- Variables are created using assignment statements.
- Constant names are usually written in uppercase letters.
- Identifiers must follow Python naming rules.
- Common data types include `int`, `float`, `str`, and `bool`.
- Comments begin with the symbol `#`.
- `input()` receives information from the user.
- `print()` displays information.
- Operators and expressions are used for calculations and comparisons.

It is not necessary to memorize all the syntax immediately. The main goal is to recognize these elements and become comfortable with reading and running simple Python programs.